

Fase 1.

Objetivo:

- Utilizar estructuras lineales y algoritmos analizados en clase en el contexto de un proyecto
- Desarrollar algoritmos eficientes que utilicen dichas estructuras lineales

Enunciado:

Queremos diseñar e implementar un sistema que permita simular un buscador web. Concretamente:

- El usuario introducirá una palabra clave. El sistema imprimirá por pantalla la URL de aquellas webs que tengan dicha palabra clave

El usuario tiene que poder obtener el resultado de manera inmediata. Por este motivo, las webs asociadas a cada palabra clave deberán ser computadas de antemano.

Tareas:

- a) Crear las clases necesarias para representar el dominio, **utilizando estructuras de datos lineales** para su implementación.
- b) Desarrollar los siguientes métodos y determinar el orden temporal de cada uno de ellos. Desarrollar también cualquier otro método que sea necesario para completar la implementación y determinar su orden temporal.

En la clase **ListaWebs**:

```
/**
 * Añade una web a la lista
 * @param web: la web a añadir
 * PRE: web no está en la lista
 */
public void anadirWeb(Web web);

/**
 * Añade un enlace a una web de la lista
 * @param idWebOrigen: id de la web de origen
 * @param idWebDestino: id de la web de destino
 * PRE: las webs con id idWebOrigen e idWebDestino están en la lista
 */
public void anadirEnlace(int idWebOrigen, int idWebDestino);
```

En la clase **Internet**:

```
/**
 * Carga las webs contenidas en el fichero indicado
 * @param nomFich: nombre del fichero que contiene las webs
 */
private void cargarWebs(String nomFich); //Ver ayuda en siguiente
                                         apartado

/**
 * Carga los enlaces contenidos en el fichero indicado
 * @param nomFich: nombre del fichero que contiene los enlaces
 */
private void cargarEnlaces(String nomFich); //Ver ayuda en siguiente
                                             apartado

/**
 * Inicializa la clase cargando las webs y los enlaces
 * @param nomFichWebs: nombre del fichero que contiene las webs
 * @param nomFichEnlaces: nombre del fichero que contiene los enlaces
 */
public void inicializar(String nomFichWebs, String nomFichEnlaces);

/**
 * Dado un string que contiene una palabra, imprime por pantalla las webs
 * que tienen dicha palabra clave
 * @param sPalabra: string con la palabra
 */
public void buscadorWeb(String sPalabra);
```

En la clase **ListaPalabras**:

```
/**
 * Añade una palabra a la lista
 * @param palabra: palabra a añadir
 */
public void anadirPalabra(Palabra palabra);

/**
 * Busca una palabra en la lista y la devuelve
 * @param sPalabra: texto de la palabra a buscar
 * @return la Palabra (si está en la lista), null en caso contrario
 */
public Palabra buscarPalabra(String sPalabra);
```

En la clase **Diccionario**:

```
/**
```

```

* Carga el diccionario desde el fichero indicado
* @param nomFich: nombre del fichero que contiene el diccionario
*/
private void cargarPalabras(String nomFich);

/**
* Asigna a cada palabra del diccionario las webs a las que hace
* referencia
* Pre: Internet y el diccionario ya están cargados
*/
private void computarWebsDePalabras(); //Ver ayuda en siguiente apartado

/**
* Carga el diccionario desde el fichero indicado y asigna a cada palabra
* del diccionario las webs a las que hace referencia
* Pre: Internet ya está cargado
* @param nomFich: nombre del fichero que contiene el diccionario
*/
public void inicializar(String nomFich);

/**
* Busca una palabra en el diccionario y la devuelve
* @param sPalabra: texto de la palabra a buscar
* @return la Palabra (si está en el diccionario), null en caso contrario
*/
public Palabra buscarPalabra(String sPalabra);

```

- c) Completar la clase **AplicacionWeb** (descargarla de eGela) para que:
- Obtenga e inicialice las instancias de Diccionario e Internet
 - Cuando el usuario escoja la opción 1 del menú, le pida una palabra e imprima por pantalla las URL de las webs que contienen dicha palabra clave.

Ayuda para la implementación:

Ayuda para cargarWebs y cargarEnlaces:

Se recomienda leer una a una las líneas del fichero. Para cada línea:

- Se pueden eliminar los espacios en blanco finales (si los hubiera) mediante el método `String trim()`.
- Se puede dividir la línea en sus diversos componentes utilizando el método `String[] split(String regex)`. Concretamente, `split` divide un string en base a las ocurrencias de la expresión regular que se le pasa como parámetro, guardando cada

componente en una posición del array resultado. En el siguiente ejemplo tenemos un string con los puntos cardinales separados por una coma. Para dividirlos, utilizamos `split` indicando que la expresión regular es la coma.

```
String puntosCardinales = "norte,sur,este,oeste";  
String[] arrayPuntos = puntosCardinales.split(",");  
System.out.println(arrayPuntos[1]); //Imprime sur
```

¡OJO! Para indicar que queremos dividir en base a un conjunto de espacios en blanco, la expresión regular es `"\\s+"`.

Ayuda para computarWebsDePalabras():

Existen diferentes opciones para computar las webs de las palabras del diccionario, pero no todas son eficientes. Se recomienda:

- En cada URL:
 - Para cada substring de 4 caracteres consecutivos que se puede extraer de la URL, **mirar de manera eficiente** si dicho substring está en el diccionario, y en caso de que así sea, asignar la web a la palabra correspondiente.
 - Hacer lo mismo para los substrings de tamaño 5, 6, ..., 10 caracteres

Entregables:

- Código completo
- Documentación que incluya:
 - Portada, con nombres de los miembros del grupo y fecha
 - Diagrama de clases completo
 - Por cada método implementado:
 - Algoritmo
 - Casos de prueba considerados
 - Orden temporal

Fecha de entrega: 29 octubre de 2021